

Load and MLP-Aware Thread Orchestration for Recommendation Systems Inference on CPUs

Rishabh Jain
The Pennsylvania State University
University Park, PA, USA
rishabh@psu.edu

John Kalamatianos
Advanced Micro Devices, Inc.
Boxborough, MA, USA
john.kalamatianos@amd.com

Teyuh Chou
Advanced Micro Devices, Inc.
Santa Clara, CA, USA
teyuh.chou@amd.com

Gabriel H. Loh
Advanced Micro Devices, Inc.
Bellevue, WA, USA
gabriel.loh@amd.com

Onur Kayiran
Advanced Micro Devices, Inc.
Bristol, RI, USA
onur.kayiran@amd.com

Mahmut T. Kandemir
The Pennsylvania State University
University Park, PA, USA
mtk2@psu.edu

Chita R. Das
The Pennsylvania State University
University Park, PA, USA
cxd12@psu.edu

Abstract

Recommendation models can enhance consumer experiences and are one of the most frequently used machine learning models in data centers. The deep learning recommendation model (DLRM) is one such key workload. While DLRMs are often trained using GPUs, CPUs can be a cost-effective solution for inference. Therefore, optimizing DLRM inference for CPUs is an important research problem with significant business value.

In this work, we identify several shortcomings of existing DLRM parallelization techniques, which can include load imbalance across CPU chiplets, suboptimal core allocation for embedding tables, and inefficient utilization of memory-level parallelism (MLP) resources. We propose a novel thread scheduler, called “Balance,” that addresses those shortcomings by (1) minimizing core allocation per embedding table to maximize core utilization, (2) using MLP-aware task scheduling based on the characteristics of the embedding tables to better utilize memory bandwidth, and (3) combining work stealing and table reordering mechanisms to reduce load imbalance across CPU chiplets. We evaluate Balance on real hardware with production DLRM traces and demonstrate up to a 1.67× higher speedup over prior state-of-the-art DLRM

parallelization techniques with 96 cores. Further, Balance consistently achieves 1.22× higher performance over a range of batch sizes.

CCS Concepts: • Computer systems organization → Multicore architectures; • Computing methodologies → Modeling and simulation; Artificial intelligence.

Keywords: Recommendation Systems, Embedding Bag Kernel, Parallelization, Memory Bound, Multi-cores, Thread Scheduling, Load Balancing

ACM Reference Format:

Rishabh Jain, Teyuh Chou, Onur Kayiran, John Kalamatianos, Gabriel H. Loh, Mahmut T. Kandemir, and Chita R. Das. 2025. Load and MLP-Aware Thread Orchestration for Recommendation Systems Inference on CPUs. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '25)*, March 30–April 3, 2025, Rotterdam, Netherlands. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3676641.3716003>

1 Introduction

Recommendation systems have transformed how consumers find products among a growing number of choices, specifically in areas such as e-commerce, media and entertainment, and online advertising [2, 17, 33, 34, 42, 48]. The market for recommendation systems generates billions of dollars in revenue [10, 35] and is expected to grow at a 36.3% rate from 2024 to 2030 [15]. Deep learning recommendation models (DLRM) [41] enable recommendation systems by working with continuous and categorical features, where large amounts of data for these features are collected from many users and then accessed frequently when processing inference requests. DLRMs require large memory capacities to accommodate the large model footprint, and they need high

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. ASPLOS '25, Rotterdam, Netherlands

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1079-7/2025/03

<https://doi.org/10.1145/3676641.3716003>

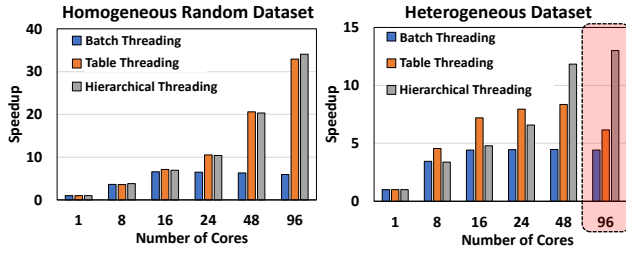


Figure 1. Performance scaling of batch threading (BT), table threading (TT), and hierarchical threading (HT) over homogeneous random and heterogeneous datasets. Reported speedup is over single core performance. Despite the significant speedup of HT over BT and TT, absolute speedup achieved over heterogeneous tables is only 13 $\times$ (highlighted) and much lower than the ideal speedup of 96 $\times$.

memory bandwidth to accommodate the memory accesses performed during inference tasks.

As with most deep learning models, DLRM training is typically performed using GPUs due to their high compute throughput and memory bandwidth [30, 39, 45, 51]. On the other hand, DLRM inference, because of latency requirements, typically uses smaller batches, which makes it amenable for execution on CPUs [11, 14, 16, 21, 25, 29, 40]. In addition, because the cost per GB of DDRx memory is typically lower than HBM memory [44], CPUs can have initial cost advantages over GPUs. These factors make CPUs attractive for recommendation model inference.

A DLRM inference task passes through four stages: embedding, bottom multi-layer perceptron, feature interaction, and top multi-layer perceptron, as shown in Figure 2. The embedding stage is known to be the most memory-intensive stage [11, 14, 16, 20, 21]. The other three non-embedding stages are compute bound. During inference, the entire recommendation model is loaded once into main memory and queries by multiple users are batched. Each inference request generates top-K recommended items to the users serviced by each batch.

The embedding stage accounts for the majority of execution time [13, 20, 21, 40]. Furthermore, it is a latency-critical stage because of tight user-facing latency requirements [5, 14, 25]. The embedding stage consists of a nested loop that loads data in vector registers from various rows of large arrays/tables and performs additions. Given the irregular memory traffic required by the embedding stage, performance benefits are limited by increasing the compute resources of CPU processors. Prior works [21, 40] investigated parallelization and thread scheduling techniques to optimize the embedding stage in the context of CPUs. The state-of-the-art CPU parallelization technique for DLRM inference [40], called hierarchical threading (HT), aims to optimize the embedding stage by mapping the work of each

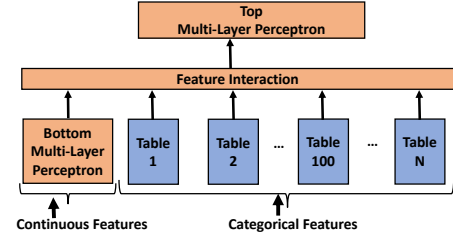


Figure 2. DLRM architecture consists of four stages: bottom multi-layer perceptron, embedding, feature interaction, and top multi-layer perceptron.

table on a separate CPU chiplet, also called core complex dies (CCD), for some architectures. The performance benefits of this approach over prior techniques are mostly due to lower embedding table data replication across private caches, absence of data replication across per-CCD shared L3 caches, lower capacity contention at the per-CCD shared L3 caches, and reduced load imbalance across threads. However, we find that HT does not effectively utilize all of the cores in the system.

Figure 1 shows the performance scaling of batch threading [19, 32, 40] (BT), table threading [40] (TT), and HT as the number of cores increases, for two different datasets (i.e., homogeneous random tables and heterogeneous tables, which are representative of production tables). As we can observe, HT can provide a 13 $\times$ speedup with the heterogeneous dataset using 96 cores. There remains a significant performance gap between the achieved 13 $\times$ speedup compared to the ideal speedup of 96 $\times$. A detailed discussion is presented in Section 2.2.

We analyze HT and identify three key opportunities. First, due to the heterogeneous embedding table structure, some CCDs are assigned more work compared to others. This *load imbalance*¹ across CCDs can cause under-utilization of compute and cache resources. Second, core utilization can be low because many of the tables do not have enough associated work to keep all the cores in a CCD busy. Third, tables are not only heterogeneous in terms of their total work, but also in terms of their data reuse characteristics. This means that some CCDs utilize their cache capacity better while they under-utilize their memory-level parallelism (MLP) resources such as miss status handling registers (MSHRs), while other CCDs saturate their MLP and leave cache capacity underutilized. As a result, execution achieves *lower MLP utilization* and thus lower memory bandwidth, which in turn limits the performance of the memory-bound embedding stage. To our knowledge, ours is the first work to study the architectural implications when executing HT with heterogeneous embedding tables and to point out the inefficiencies leading to multi-core underutilization.

¹HT has low load imbalance among threads in the same CCD.

To quantify the impact of table heterogeneity on *core allocation*, we introduce a heuristic called *table load (TL)* that approximates the execution time of a table. We use TL to categorize tables as heavy-weight, medium-weight, or light-weight. Experimental analysis reveals that light-weight tables experience higher thread-scheduling overheads [46] and benefit less from more cores compared to heavy-weight tables. To quantify the impact of table heterogeneity on *MLP*, we perform a study with more MLP resources (i.e., L3 MSHRs) while other compute resources stay the same and observe significant speedup. Next, we introduce a new metric called *weighted MLP (WMLP)* that approximates the MLP demand of each table.

Based on these analyses, we develop three mechanisms to exploit the identified opportunities. First, to address *low core utilization*, instead of always executing the work associated with a table using all cores of a CCD, we propose to statically allocate a subset of the cores for each table. Second, to improve *MLP* utilization, we use two work queues (one for high- and one for low-WMLP tables) to co-locate tables from different queues on the same CCD. Finally, we employ a combination of work stealing and table reordering mechanisms to reduce *load imbalance*. To our knowledge, the three ideas (low-core grouping, MLP-aware scheduling, work stealing and table reordering) have not been proposed by prior works for DLRM inference.

The combination of these three mechanisms results in a 1.67× performance improvement over HT. Our analyses and evaluation demonstrate that the performance of DLRM inference on CPUs can be significantly improved with better utilization of core, cache, and MLP resources in the SoC.

Algorithm 1 Embedding stage kernel consists of five loops. The two outer loop nests are implemented in Python [37], while the three inner loop nests are implemented inside the C++ embedding bag operator [38].

```

1: for  $i$  in  $num\_batches$  do
2:   for  $j$  in  $num\_tables$  do
3:     for  $k$  in  $batch\_size$  do
4:       for  $l$  in  $pooling\_factor$  do
5:          $vec.ld\ accm;$ 
6:         for  $m$  in  $embedding\_dim$  do
7:            $vec.ld\ row\_block;$ 
8:            $vec.add\ accm,\ accm,\ row\_block;$ 
9:         end for
10:         $vec.st\ accm;$ 
11:       end for
12:     end for
13:   end for
14: end for

```

2 Embedding Stage Parallelization Inefficiencies

This section first discusses the DLRM software pipeline and the quantitative properties of the DLRM inference production traces. Then we discuss prior DLRM parallelization approaches and quantify their limitations in multi-core scaling. To better understand the scaling issues, this section analyzes limitations of HT by exploring the following questions: (1) How prevalent is load imbalance across CCDs? (2) How well does the work of each embedding table scale with respect to the number of cores? (3) How effectively are a CCD's MLP capabilities being utilized? The answers motivate the solutions adopted by Balance.

2.1 Overview of Embedding Stage Computation and Dataset

Algorithm 1 shows the embedding stage kernel in pseudo code. Batches are formed from arriving queries where each batch executes work associated with each table and contains a batch size number of samples. Each sample executes some pooling operations where a number of embedding vectors associated with the input indices are first looked up (SIMD load operation) in an embedding table and then aggregated (SIMD add operation). This process creates a load-to-add dependency chain. The volume of data loaded per batch is a function of the model and dataset parameters. For example, with 100 tables, a batch size of 1024, and a static pooling factor of 150, a total of 7.5GB of data will be loaded per batch. This substantial amount of data loading has to be completed within tens of milliseconds, making this stage very memory-intensive, and thus demanding efficient usage of the memory hierarchy.

Next, we discuss the properties of the DLRM production traces. Prior research efforts targeting inference [11, 14, 16, 21, 25, 29, 40] have focused on input datasets that are limited in various dimensions, involving a small number of embedding tables, a low pooling factor (number of lookups per user), homogeneous (in accessing table rows) tables, or a high concentration of hot table rows. In contrast, the production traces from Meta [31] contain hundreds of embedding tables and observe significant variations in table properties such as the number of lookups per input, row sizes, and memory access patterns. We refer to this industrial dataset as the heterogeneous dataset.

Figure 3 describes various non-uniformities for a 110 GB sized model containing 129 tables. We observe the following: (1) Table sizes range from 100 to 5 million rows, which implies diverse memory capacity per table. For instance, tables with memory capacity under 1MB (2K rows) or 24MB (48K rows) can fully reside in the L2 or L3 caches, respectively. (2) Memory access patterns differ across tables. To quantify this, we use two metrics: unique indices and 10% coverage. Unique indices show the fraction of table rows touched by

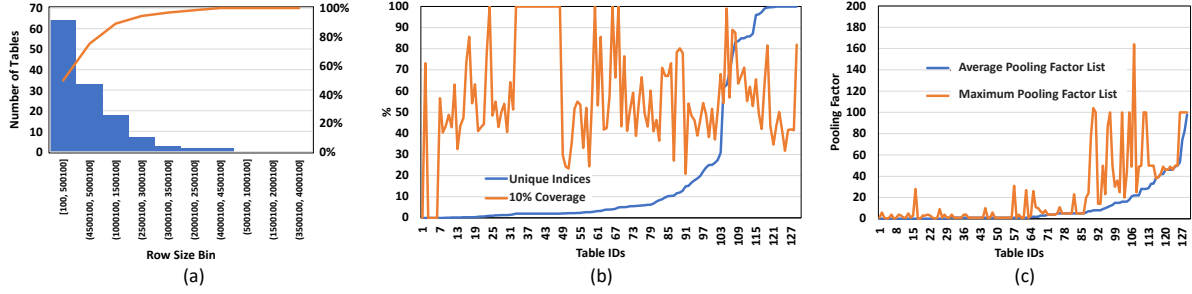


Figure 3. (a) Number of rows in various row size bins, (b) access pattern behavior for different tables, and (c) pooling factors for different tables.

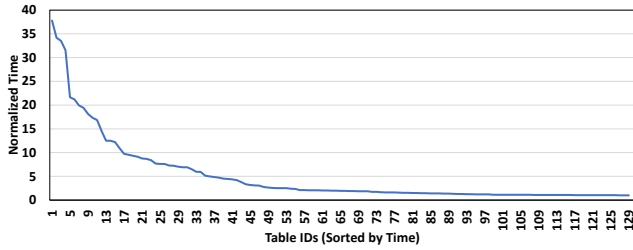


Figure 4. Embedding table latencies for different tables.

the dataset, while 10% coverage represents the percentage of lookups captured by the top 10% of the unique indices. It is evident that the production traces exhibit a wide range of hotness levels that imply varying cache utilizations across tables. (3) The maximum pooling factor across the tables ranges from zero to 164, which implies diversity in the raw amount of pooling operations across tables. For instance, a pooling factor of zero implies that there is no work associated with the table. Finally, the variations in table size, pooling factor, and memory access patterns influence the execution time per table. Figure 4 shows the sorted normalized execution time of each table. We observe that the time per table can drastically vary, by as much as 37.8 $\times$.

2.2 Limitations of Prior Parallelization Approaches

To accelerate the DLRM embedding stage, shown in Algorithm 1, Jain et al. [21] proposed parallelizing the embedding stage execution at a coarser granularity by mapping each batch to a different CPU core. However, given the trend of growing model and dataset sizes [4, 30, 39], Nair et al. [40] proposed novel ways to parallelize the work assigned to embedding tables as described in Figure 5.

BT is the default strategy used by popular frameworks such as PyTorch [32] and IPEX [19]. BT maps the work of a single embedding table (within a batch) across all cores in a SoC. Thus, only one table is processed at any given time, allowing the table to utilize the entire on-chip cache capacity. However, BT can encounter three challenges: (1) inefficient use of on-chip cache resources due to embedding

table data replication in private and shared caches and due to the presence of tables with a small number of accessed rows; (2) runtime overheads due to frequent fine-grain thread synchronization to perform reductions and thread join/fork activity; and (3) load imbalance due to straggler threads (some users accessing more embedding table data than others).

TT, proposed by Nair et al. [40], assigns the work of each table to one CPU core. In this case, for an N -core CPU, N tables are processed in parallel. Thus, TT spawns fewer threads, which reduces runtime overheads and more efficiently uses private cache capacity. However, TT can suffer from (1) cache thrashing at the L3 cache and (2) load imbalance because threads process varying amounts of work per table. TT’s load imbalance is generally much higher than BT, especially when the work of the tables with many accessed rows is mapped to one CPU core or to the cores of the same CCD simultaneously.

Finally, HT capitalizes on the advantages and mitigates the weaknesses of BT and TT. It maps the work of each table to one CCD.² In this case, for a D -CCD CPU, D tables execute in parallel. However, HT could still suffer from various issues, as we discuss next.

We study the multi-core scaling of prior schemes as shown in Figure 1. “Homogeneous random” refers to the random dataset used in recent works [6, 8, 21, 43, 54]. We observe that HT scales up to 96 cores on both datasets and performs better than BT and TT by up to 5.2 $\times$ and 1.56 $\times$, respectively. Thus, we consider HT as the most performant parallelization scheme. However, HT’s performance speedup with the heterogeneous dataset is 2.6 $\times$ lower than the speedup over the homogeneous dataset. In addition, HT’s speedup of 13 $\times$ is much lower than the ideal speedup of 96 $\times$ with all 96 cores. Therefore, there exists a substantial gap (and opportunity) between the achieved and the ideal performance scaling.

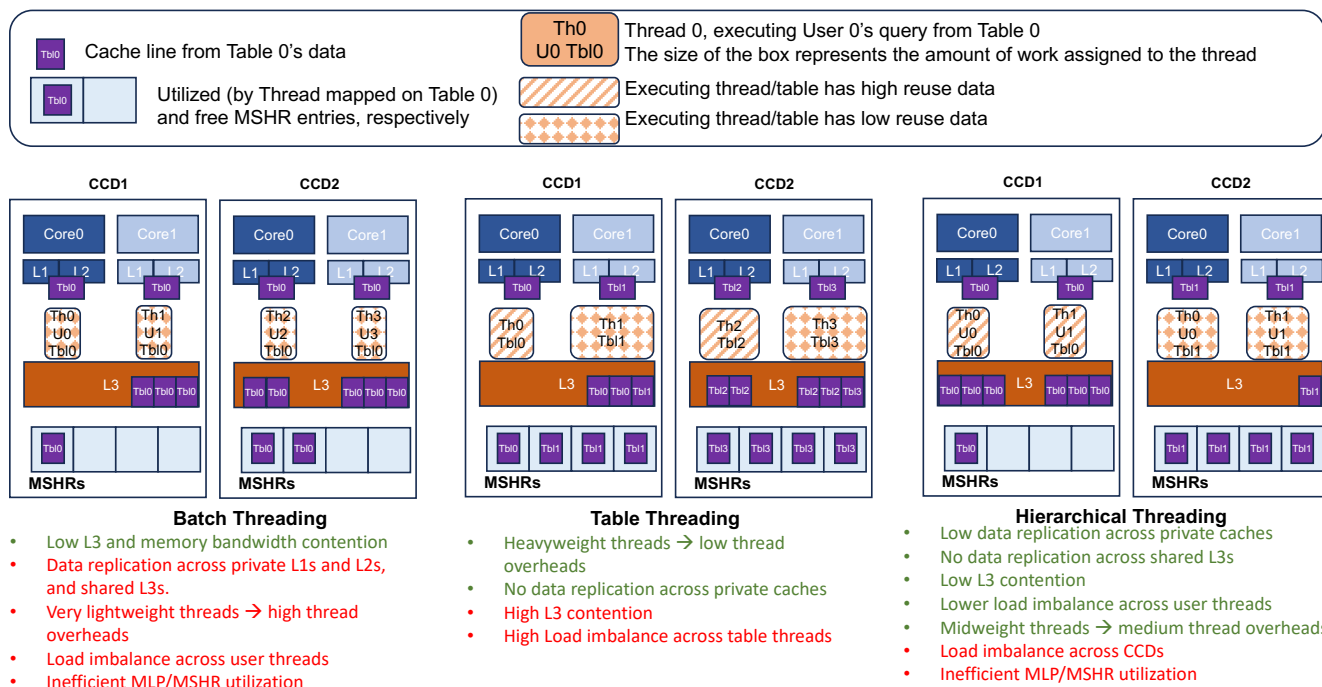


Figure 5. State of the art parallelization techniques.

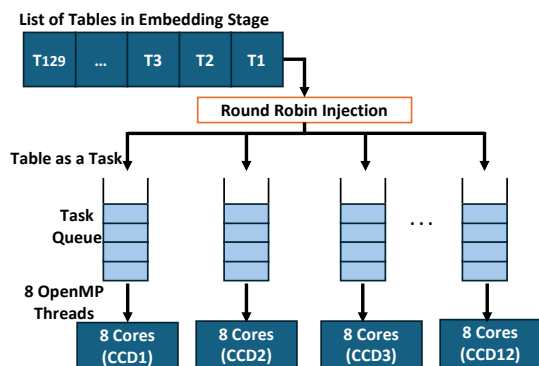


Figure 6. HT is implemented with the PyTorch thread pool. Each CCD is associated with a task queue and tables are distributed across task queues using round robin injection.

2.3 Unboxing the Design of Hierarchical Threading

To implement HT, the processing of a table is multi-threaded across CCDs using the “pytorch.jit” library [36]. Specifically, in the top-level python code (Algorithm 1), “torch.jit.fork” is used to spawn the tasks that process the work of a table, and “torch.jit.wait” is used to collect the embedding output matrix from each task. These spawned tasks are managed using PyTorch’s backend C++ thread pool implementation [38].

²In 96-core AMD 4th Generation EPYC™ Processors, a CCD is a group of eight CPU cores that share a L3 cache. Different AMD CPUs can have one or more CCDs per socket.

A task queue is coupled with a CCD to buffer the tasks to be completed by the CCD. Once a CCD receives a task, the batch amount of work associated with the table is multi-threaded across the CCD's eight cores using the OpenMP library. The OpenMP parallelization is implemented in the embedding bag operator.

Figure 6 provides an overview of HT. First, a list of tables present in the embedding stage are assigned to tasks as they enter the thread pool. Second, these tasks are enqueued into different task queues using a round-robin injector that aims to balance the distribution of tables across the queues. Third, the scheduler dequeues a task from its corresponding queue one at a time and executes the work using all of the cores of its assigned CCD.

We believe three key issues are present with the HT scheduler:

1. *Load imbalance*: Because the queues are balanced in terms of the number of tables and not the execution time for processing the tables, several heavy tables could be mapped to one task queue. Furthermore, when the task queue associated with a CCD gets empty, the CPU cores become idle despite tasks pending in other queues. This issue can lead to straggler threads.
2. *Suboptimal core allocation*: Despite the non-uniformity of work across tables shown in Figure 3, HT assigns the same number of eight cores for the work of each table. This leads to a wide variation of per-thread execution

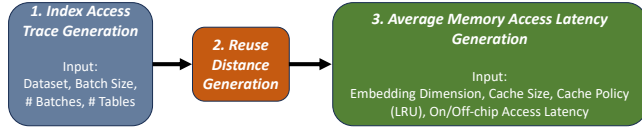


Figure 7. Reuse distance based analytical model to calculate the average memory access time of the loads accessed in an embedding table.

time and lower core utilization. Moreover, tables with less work are more vulnerable to runtime overheads.

3. *Memory level parallelism (MLP) imbalance*: When the work of a low-reuse table executes on a CCD, L3 cache MLP resources (such as MSHRs) could get oversubscribed and create backpressure on the CPU cores. Similarly, when multiple low-reuse tables execute together on different CCDs, memory bandwidth could be saturated. On the other hand, when high-reuse tables execute on one or more CCDs, L3 cache MLP and memory bandwidth resources could be underutilized.

2.4 Modeling and Characterizing Table Workloads

Because HT maps the work of each table across CCDs without considering the per-table work, it is possible that many heavy tables are mapped to one or a few CCDs, creating load imbalance issues. For example, in the worst-case workload distribution scenario, we observed that the CCD with the highest workload performs 37.8× more work than the CCD with the least amount of work, causing severe underutilization in the remaining eleven CCDs. Thus, we need to make more efficient work per table-to-CCD assignments.

For a given model and data set, the load or work per table is impacted by the embedding dimension, batch size, pooling factor, and memory access patterns. To effectively differentiate the work across tables, work can be modeled as a function of pooling factor and memory access patterns. Pooling factor defines the gather-reduce operations per table (Algorithm 1). However, pooling operation time depends on the latencies of the underlying loads, which could be significantly high with low-reuse table row access patterns [21]. To quantify the work associated with a given table, temporal locality can be used to estimate the load instruction latency.

To approximate the work per table, we take inspiration from Jain et al. [21] and introduce a heuristic called the table load (TL) that is the product of average pooling factor (APF) and average memory access time (AMAT). For a given batch size, APF is calculated by averaging the total table row indices accessed in the batch. Figure 7 describes how to calculate AMAT: (1) track table row indices as a proxy for unique memory addresses visited and collect a trace of all touched table row indices in the first batch; (2) calculate reuse distance over this trace by tracking the number of unique row indices touched in between two instances of the same

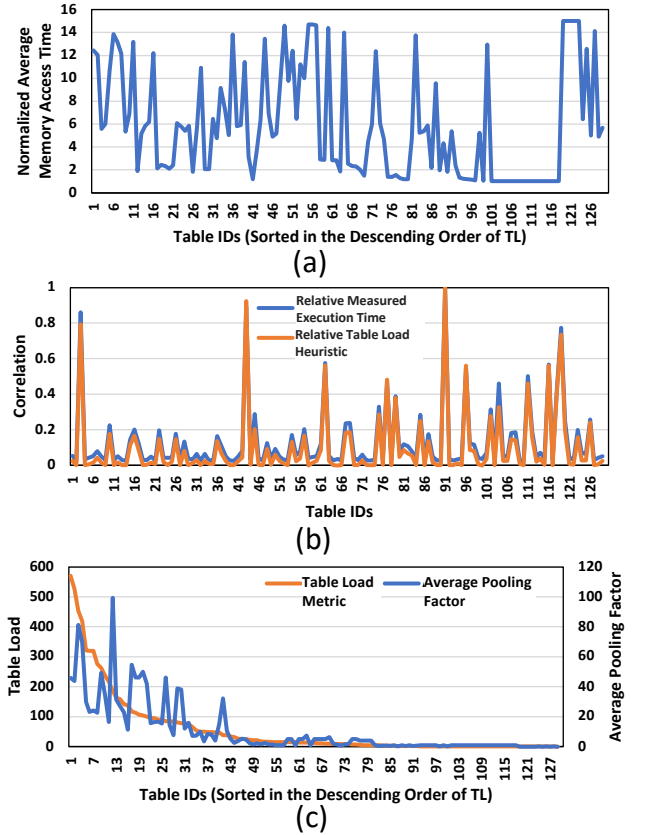


Figure 8. (a) AMAT across tables derived using our analytical model. (b) High correlation is observed between TL and measured execution time. (c) When comparing the decreasing trend of TL with pooling factor, we note that pooling factor can incorrectly represent work per table.

row index; (3) calculate AMAT by marking each row index access as a cache hit or miss, and attaching an average cost expressed in time units. We assume an embedding dimension of 128 elements (512B) and an average cost of 30 time units for a L3 hit and 450 time units for a L3 miss based on memory latency measurements on actual silicon [7], thus giving a ratio of 1:15 for on-chip versus off-chip access. Further, we identify row index accesses as hits or misses by simulating a fully-associative, 32MB L3 cache with LRU replacement. This effectively defines the maximum recorded reuse distance to be $32\text{MB}/512\text{B} = 64\text{K}$ row index accesses.

To illustrate the above steps, consider a sample table with an APF of 81.3. The model outputs the on-chip cache hit rate as 67.4%. Thus, AMAT is $(67.4 \times 1 + (100 - 67.4) \times 15) / 100 = 5.57$ and TL can be calculated as $81.3 \times 5.57 = 452.7$. Figure 8(a) depicts the AMAT values for a list of sample tables, and confirms the presence of heavy variation across the reuse behavior within tables. Also, Figure 8(b) compares the TL

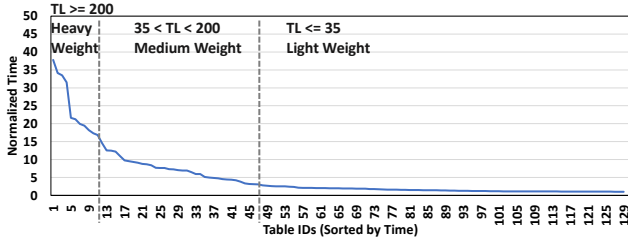


Figure 9. Using different TL metric thresholds, the embedding tables can be categorized as heavy, medium, and light-weight.

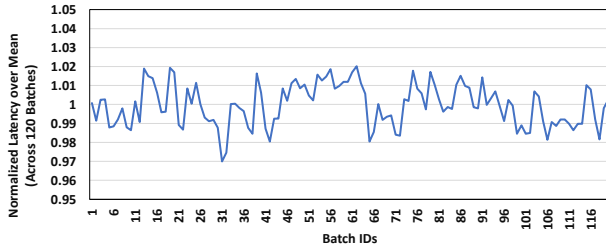


Figure 10. Deviation study of embedding stage execution time across 120 batches. Minimal deviation is observed.

metric with the measured execution time and the high correlation provides high confidence in its applicability. Notably, prior studies [39, 45] have estimated the work per table using only APF and have missed the impact of temporal locality. In contrast, we account for both and that enables us to more accurately compare the work across tables. Specifically, Figure 8(c) illustrates that TL and APF values vary across tables. For example, while the execution time for Table ID 3 is only 56% higher than Table ID 6, the APF of the former is $3.6\times$ larger than the latter, resulting in significant inaccuracies when comparing the work across tables. AMAT more closely approximates the work per table.

Finally, we cluster tables based on their TL. Figure 9 sorts the tables in descending order of their execution time and categorizes them into heavy, medium, and light-weight classes using TL. This categorization helps reduce load balancing across CPU cores by avoiding straggler threads assigned the work of heavy tables as described later in Section 3.

Discussion on applicability of TL: We believe that it is sufficient to derive the TL value from profiling a few batches because we note that table properties, like execution time, APF, and AMAT, exhibit minimal variation across batches. Figure 10 confirms the robustness of TL across batches. If the DLRM embedding tables are updated, which may happen every few hours after serving hundreds of thousands of batches [29, 30], the TL metric can be very quickly regenerated over the table row access patterns of new batches. Furthermore, the TL metric is more accurate compared to

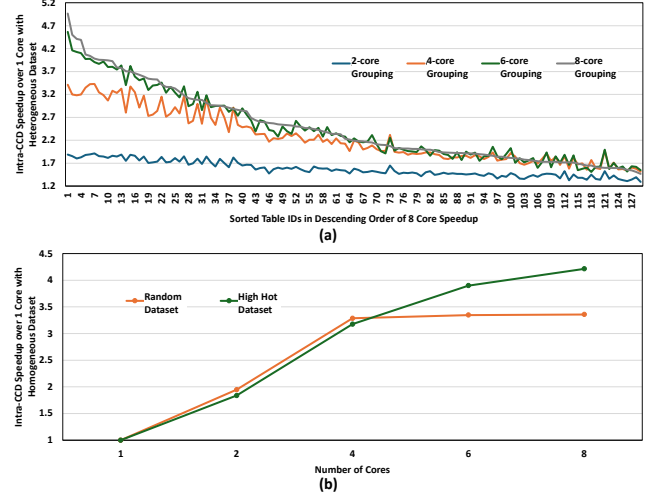


Figure 11. Characterization of intra-CCD multi-core scaling of embedding tables. (a) Work of each table scales differently with the number of cores, and only a few tables benefit from 8 or 6 cores. (b) Unlike a high reuse table whose work benefits from using up to 8 cores, the work of a low reuse table stops scaling after 4 cores.

other profiling techniques [52, 53], which depend on silicon-specific profiles such as actual cache hit rates or incur additional runtime overheads. The TL metric approximates work per table based on properties of the application and high-level table row-level reuse profiles.

2.5 Per Table Multi-Core Scaling Behavior

The previous section studied the per-table heterogeneous behavior, especially in terms of execution time and reuse of per table row accesses and found wide variations across tables. We now study how the work per table scales with the number of cores per CCD because HT maps the work of each table to all cores per CCD (8 cores in our target SoC). Figure 11 shows the scaling of embedding tables over various core counts. We note that five tables benefit from using all eight cores, 35 tables benefit from six cores, and the remaining long tail of 89 tables benefit from two or four cores. Thus, only a small fraction of tables can fully utilize all eight cores, confirming that prior schemes tend to over-provision the number of cores for the majority of the tables.

To identify the minimum number of cores needed for optimal scaling of a table, Figure 12 plots the realized speedup of per-table work using 2, 4, and 8 cores. The tables, in the x-axis, are sorted in descending order of their TL. We note that tables with larger TL benefit from more cores. In other words, there is a correlation between speedup and TL. Thus, we can use the TL metric to cluster tables based on the optimal number of cores, as shown in Figure 1.

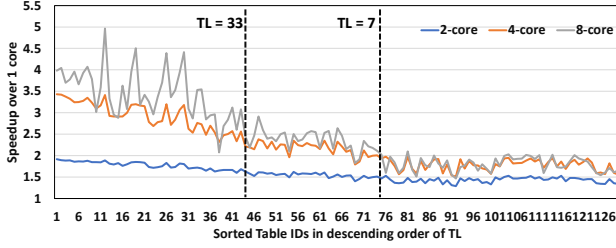


Figure 12. Speedup using 2, 4, and 8 cores over tables sorted in descending order of work per table (TL).

Core Count	TL > 33	33 ≤ TL < 7	TL ≤ 7
2	1.6 – 1.9x	1.5 – 1.6x	1.3 – 1.5x
4	2.5 – 3.5x	2 – 2.5x	1.5 – 2x
8	3 – 5x	2 – 3x	1.5 – 2x

Table 1. TL can predict the range of performance scaling for any given table.

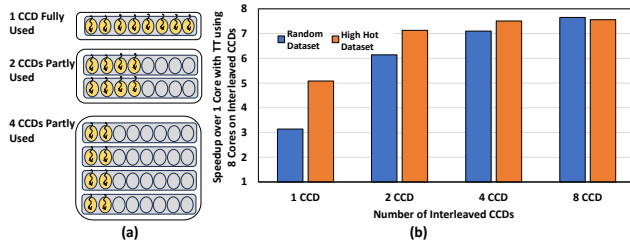


Figure 13. Eight thread execution using multiple CCDs with TT over different memory reuse patterns.

2.6 Microbenchmarking Memory Level Parallelism

To complete the gather-reduce tasks as part of the pooling operation, the CPU cores are essentially dependent on the rate of data supplied by the memory hierarchy. This highlights the importance of understanding how memory access patterns coupled with a given memory hierarchy organization impacts multi-core scaling. Figure 11(b) evaluates the intra-CCD multi-core scaling for two extreme table row access patterns. We observe that the tables with high reuse behavior (high hot data set) scale up to eight cores. However, tables with low reuse behavior (random data set) do not scale after four cores. This suggests a resource limitation in the L3 cache’s ability to exploit the MLP needed to service the off-chip requests generated after more cores are tasked to execute the work of the tables.

To highlight the limited L3 cache MLP per CCD, we conduct a systematic experiment that isolates the number of cores from the memory hierarchy. We do this by fixing the number of threads and cores to eight and vary the number of CCDs to increase the available memory hierarchy for the

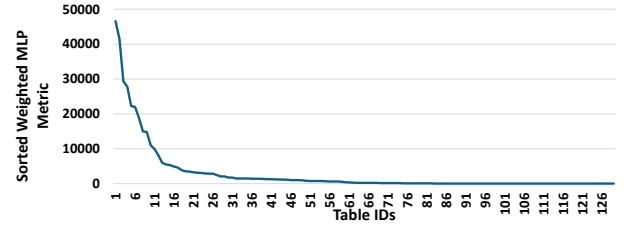


Figure 14. Weighted memory-level-parallelism (WMLP) metric models the amount of MLP demand of each table.

eight threads. Specifically, we utilize the eight cores by distributing the threads across CCDs. Figure 13(a) highlights how eight threads can be distributed across one or more CCDs. It is important to note that even though the number of cores remains the same, the memory hierarchy observed by the threads highly differs. For example, when eight threads are using a single CCD, they share the L3 cache capacity and MSHRs. However, when eight threads are mapped to one core per each of eight different CCDs, each thread uses the entire L3 cache capacity and MSHRs which can minimize the number of misses and maximize the number of simultaneous miss requests per thread.

Given that any two embedding tables do not share data, we use TT for the 8-core scaling experiment. This prevents any sharing of data among threads of the same table. Figure 13(b) shows the multi-core scaling of TT for low-reuse and high-reuse tables. Speedups up to 7.7× are achieved over the low-reuse case, confirming that higher L3 MLP helps in increasing the data supply rate to the cores. This is mainly due to an increase in outstanding L3 cache misses because multi-core scaling for the high-reuse tables, which have lower memory footprint and better cache hit rates, saturates with two cores.

Because MLP requirements vary per table, the MLP resources per CCD could get oversubscribed. Thus, it becomes useful to predict the MLP demand per table and leverage this information by the thread scheduler. We develop a new metric called Weighted-MLP (WMLP) that captures the MLP required for a table. It is defined as the product of TL and L3 cache hit ratio (%). Figure 14 shows the WMLP variation across tables, which ranges from 0 to 46600. Tables with high pooling factors and high cache miss rates generate a large WMLP. On the other hand, tables with low pooling factors and low cache miss rates have a small WMLP. A few tables, which have zero pooling operations, exhibit a WMLP value of zero. Overall, significant variations in the WMLP are observed.

2.7 Key Architectural Insights for CPU Multi-Core Scaling inefficiencies

Based on the above characterization study, the following architectural insights can be drawn:

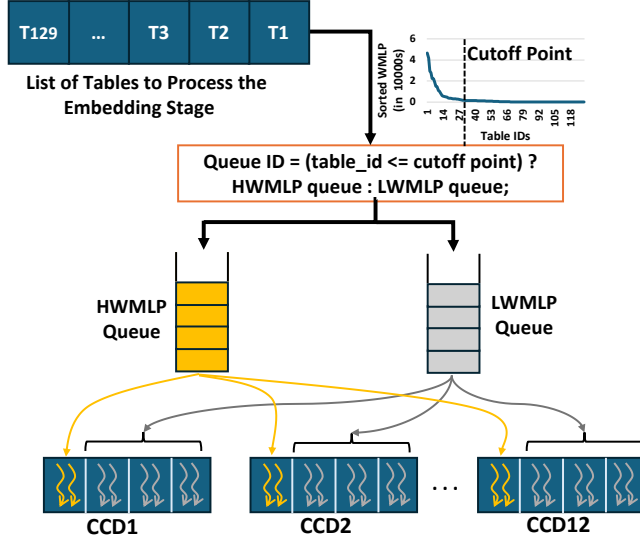


Figure 15. Balance is demonstrated for two-core grouping. Tables are injected into the task queues in decreasing WMLP order. Two task queues are used to separately schedule the work of heavy (HWMLP) and light (LWMLP) tables based on a cutoff point. At any point in time, each CCD is assigned only one HWMLP table and one or more LWMLP tables. Work stealing improves load balancing by allowing a CCD to seek work from a secondary queue after a primary queue gets empty.

- Drastic differences in the work per table are possible, which can lead to load imbalance across CCDs. The work per table can be accurately modeled using the TL heuristic, which could be used to better map the tables onto cores.
- Allocating eight cores per table is excessive for the majority of tables. Instead, an appropriate amount of cores can be predicted based on the TL of each table. Specifically, two or four cores per table could help better utilize the cores.
- Available MLP in a given CPU impacts the multi-core scaling. The WMLP metric predicts the MLP demand of each table and can be used to distribute low-reuse tables across CCDs to improve scaling.

3 Load- and MLP-Aware Thread Scheduling

In this section we describe our novel thread scheduler called “Balance.” We modify the original HT-based thread pool implementation (Figure 6) by adding WMLP-based table ordering, MLP-conscious task scheduling using two task queues across all CCDs per SoC, low core grouping per table and work stealing. Figure 15 shows an overview of our proposed thread scheduler. Note that the embedding stage execution

is bulk synchronous and therefore its performance is dictated by the total completion time of all tables and not by the completion time of individual tables.

Low-core grouping. Figure 11 showed that the number of cores, beyond which performance of per-table work does not scale, varies across tables. If the work per table does not scale up to all cores in a CCD, the unused cores can be assigned to work on other tables, enabling higher resource utilization per CCD. Low-core grouping allows per-table work to be mapped to the minimum number of cores needed to scale performance. In order to keep the HT scheduler complexity down, we propose a simplified static allocation of either two or four cores per table. Figure 15 shows a two-core allocation for all the tables. Once a new task is received, the work associated with the table is mapped onto the designated number of cores (two or four) by leveraging OpenMP threads. Compared to eight-core grouping in HT, since the work per thread is two and four times higher in the four-core and two-core grouping, respectively, we expect lower runtime overheads compared to HT.

MLP-aware scheduler. Embedding tables exhibit different MLP requirements as noted in Section 2.6; therefore, for each inference task, we separate the tables into two groups: HWMLP and LWMLP. Each group is sorted in descending WMLP order. The scheduler employs two queues. Tables from the HWMLP (LWMLP) group are injected into the HWMLP (LWMLP) queue. The work for each table is mapped to two or four cores based on its predefined core grouping. The scheduler first starts mapping the HWMLP tables across the CCDs in a round robin fashion, limiting a maximum of one HWMLP table per CCD. This limit typically leaves enough MLP capacity for LWMLP tables to be co-located on the same CCD. Figure 15 shows an example with one HWMLP table co-located with three LWMLP tables in every CCD. This approach provides a good tradeoff between maximizing the number of tables being scheduled simultaneously and utilizing MLP resources per CCD. Note that if we fully utilize MLP resources per CCD, we also maximize memory bandwidth usage across the SoC because memory bandwidth depends on the in-flight requests sent by all the CCDs to main memory.

The MLP-aware scheduler has several advantages over the baseline HT scheduler. First, it uses only two queues instead of a number of queues equal to the number of CCDs in the SoC. This reduces scheduling overheads, which is now independent of the number of cores per CCD. Second, low-core grouping maximizes core utilization. Third, MLP-conscious scheduling allows for higher utilization of MLP and on-chip cache resources per CCD. Fourth, the combination of MLP-conscious table scheduling and low-core grouping improves load balancing compared to the baseline HT because it allows for a more balanced distribution of work across the CCDs. In contrast, the HT scheduler maps each table to a CCD independent of its MLP or compute requirements. Fifth,

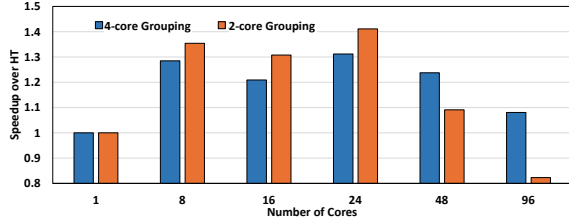


Figure 16. Speedup of Balance with two and four-core grouping over HT using various core counts and batch size of 1024.

the Balance scheduler avoids scheduling multiple HWMLP tables onto the same CCD, therefore reducing the probability of straggler threads compared to the HT scheduler.

Work Stealing. Even though MLP-based table scheduling balances the work assignment across the two task queues, there can still exist cases where load imbalance may arise if, in one inference task, a HWMLP group contains far fewer tables than the LWMLP group. This scenario can lead to cases where one of the task queues is empty while the other still has more work to schedule. In order to address this, we employ work stealing. When one of the queues (HWMLP or LWMLP) is empty, we steal the work of a table from the other queue. Thus, the scheduler will continue scheduling work from both queues.

Synergy between Balance scheduler features. We believe each of the above proposed features can complement each other. To understand this, we consider three pathological scenarios: (1) if a DLRM inference task includes a skewed distribution of embedding tables (e.g., HWMLP versus LWMLP), MLP-conscious table reordering and work stealing ensures that Balance efficiently distributes tables across cores, lowering the chances of straggler threads; (2) if a DLRM inference task must process embedding tables with very small TL values (similar to the tail in Figure 12), low-core grouping ensures that cores remain utilized; (3) if an inference task processes embedding tables with either very high-reuse or very low-reuse memory accesses, MLP-aware scheduling ensures that caches and MLP resources remain highly utilized. Thus, we expect Balance to perform better in all of the above challenging scenarios compared to a HT scheduler.

4 Methodology

Model and Datasets: We use an embedding dimension of 128 and a batch size of 1024 (unless mentioned otherwise). Our experimental evaluation includes two sets of traces classified as “homogeneous” and “heterogeneous.” The homogeneous dataset exhibits uniform properties across tables such as table size, batch size, pooling factor, and embedding dimension for each inference task. More specifically, we consider an inference task with 96 embedding tables, 500K rows

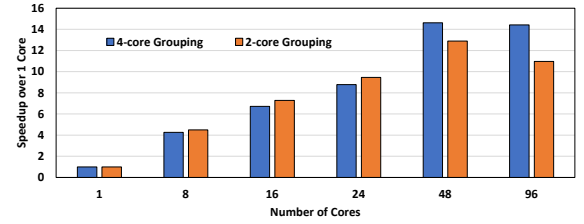


Figure 17. Speedup scaling for Balance with two and four-core grouping over single core Balance and batch size of 1024.

System	Sockets	1
	PCIe	128 lanes Gen5
SoC	Cores per SoC	96
	CCDs per SoC	12
	Memory BW	460GBps across 12 DDR5 channels
	Memory Capacity	768GB per socket
	TDP	360W
CCD	Cores per CCD	8
	L3 per CCD	32MB
Core	L1 Cache (instr/data)	32KB/32KB
	L2 Cache	1MB
	L3 Cache	4MB
	DTLB entries (L1/L2)	2048/3072
	Frequency	3.5GHz

Table 2. 96-core AMD 4th Generation EPYC Processor Configuration [3].

each, and 150 lookups per user (pooling factor). The homogeneous datasets are obtained from [21] and are categorized based on reuse as high hot, medium hot, low hot, and random. The heterogeneous datasets are production traces from Meta [31] and possess non-uniform properties across tables such as table size, pooling factor, and memory access patterns, as described in Section 2.1. For a batch size of 1024, a total of 128 batches are run, where the first eight batches are marked as cold execution due to initial function calls and performance is averaged over the later 120 warm batches.

Hardware: Table 2 describes the hardware configuration of our system setup. In all of our evaluations, performance is studied for a one-socket processor. A core complex die (CCD) refers to a group of eight cores sharing a L3 cache in our target CPU.

Software: PyTorch (v1.13.1) is source-compiled along with the ZenDNN library featuring an AVX512-based embedding bag operator implementation. GCC 11.4.0 is used for code compilation. Our setup runs under Ubuntu®/Linux® (Ubuntu 22.04.2 LTS) with the 5.19.0-32-generic kernel.

System: We enable Transparent Huge Pages (THP) and frequency boost. SMT and stream hardware prefetchers (L1 and L2) are disabled. THP is enabled to take the address translation overhead out of the critical path of the embedding bag (EB) kernel. This increases DTLB hit rates and, as a result,

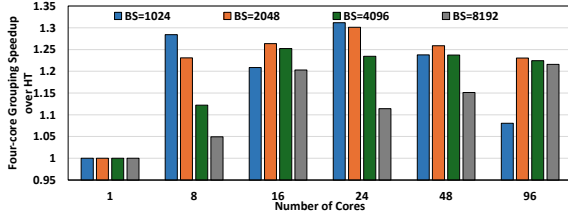


Figure 18. Speedup of Balance with four-core grouping against HT over various core counts and batch sizes (BS).

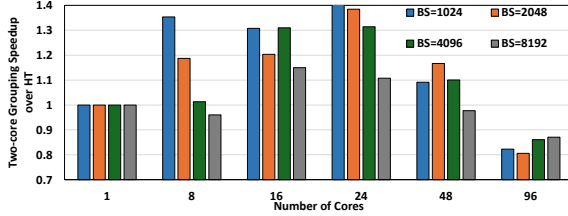


Figure 19. Speedup of Balance with two-core grouping against HT over various core counts and batch sizes (BS).

batch latency improves by 20-30%. Stream prefetchers do not help with the irregular memory traffic associated with the EB kernel. SMT does not help the EB kernel performance due to limited MLP per thread [21].

5 Evaluation

In this section, we evaluate the performance of the Balance scheduler. Given that HT is the most performant parallelization scheme for the embedding kernel of DLRM, we compare HT against Balance using two-core and four-core groupings. The two (four) core grouping allows for a maximum of four (two) tables to be processed per 8-core CCD. All evaluations are conducted using the heterogeneous datasets.

5.1 Main Results

Here we evaluate the threading schemes over a model with 129 tables. Figure 16 compares Balance and HT for a batch size of 1024. Balance with either four or two-core grouping achieves up to 1.3× and 1.41× speedup over HT for a 24-core configuration while scaling stagnates with more cores mainly due to the higher memory traffic in the data fabric (DF, network on chip) as we schedule work on more CCDs. However, the four-core grouping improves performance over HT even with 96 cores. Performance scaling over a single core with Balance (and stagnation at 48 cores) is also shown in Figure 17.

To increase the work per thread, we evaluate larger batch sizes (2048, 4096, and 8192) as shown in Figure 18 and Figure 19. Four-core grouping achieves consistent speedup over HT across all batch sizes and core counts, while two-core grouping does not always, especially with larger batch sizes.

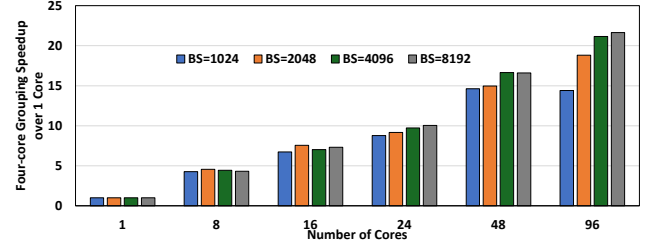


Figure 20. Four-core grouping speedup over a single core.

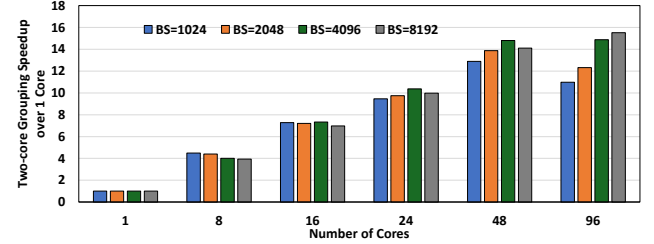


Figure 21. Two-core grouping speedup over a single core.

This is because two-core grouping allows a larger number of tables to be processed in parallel generating more L3 misses and DF memory traffic at a given core count than four-core grouping. Figure 20 and Figure 21 show similar comparisons but over a single core implementation. A higher batch size implies more work per table, and thus better amortizes thread creation/deletion overheads allowing Balance to better scale performance. Performance improvements over single core peaks at 21.65× with a 8192 batch size. Thus, Balance is able to achieve 1.67× better scaling over 96 cores compared to HT with batch size of 1024. Moreover, the absolute embedding bag kernel latency under Balance is 2.5msec, which can better meet real-time latency requirements.

In Figure 19, Balance with two-core grouping does not achieve consistent performance improvements with higher core count and batch size. A similar observation is made with table threading and 96 cores (Figure 1). This performance anomaly is because a few high-work tables are mapped to the same CCD, taking longer to complete the task. Our initial studies found that varying the cutoff point impacts performance. Thus, we expect modifying the cutoff point can prevent this behavior via better distribution of the tables over CCDs and two-core worker groups.

5.2 Sensitivity Studies

In this section, we evaluate threading schedulers over a DLRM model with 400 tables.

1) *Impact of cutoff point and work stealing:* Figure 22 shows the performance of work stealing in the Balance scheduler with a batch size of 2048 and 48 cores under various cutoff points. Without work stealing, we observe wide performance

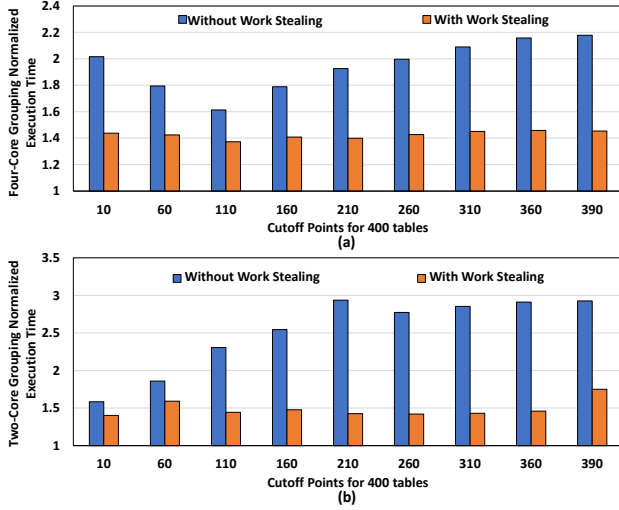


Figure 22. Impact of cutoff point and work stealing.

variations across the cutoff points. The deviation is higher for two-core grouping because the latter is more sensitive to uneven table distribution.

Performance deviation is largely reduced under work stealing because it dynamically balances the load across the workers. In addition, work stealing at the optimal cutoff point improves performance by up to $1.5\times$ and $2.06\times$ for four- and two-core grouping schedulers, respectively. We also note that work stealing was invoked 35.4 and 71.2 times per batch by the four- and two-core grouping Balance schedulers, respectively. To keep the load balanced, the two-core grouping Balance scheduler coupled with a 1:3 worker ratio invokes work stealing more often than the four-core grouping.

2) *Boost in embedding stage performance:* Figure 23 compares the performance of Balance over HT for batch sizes of 1024 and 2048. Similar to Figure 18 and Figure 19, Balance continues to outperform HT, especially with the four-core grouping scheduler, especially at the highest core counts.

3) *Memory bandwidth utilization:* Figure 24 compares the memory bandwidth of HT and Balance schedulers in a 48-core and 2048 batch size configuration. Balance schedulers achieve higher memory bandwidth utilization than HT by improving cache utilization (lower traffic to DRAM and faster execution time). However, two-core grouping sees a higher memory bandwidth utilization of $1.39\times$ even though the performance improves only by $1.2\times$. Because two-core grouping schedules up to four tables per CCD, it frequently suffers from higher shared L3 cache miss rates compared to four-core grouping, resulting in higher memory bandwidth due to higher traffic to DRAM.

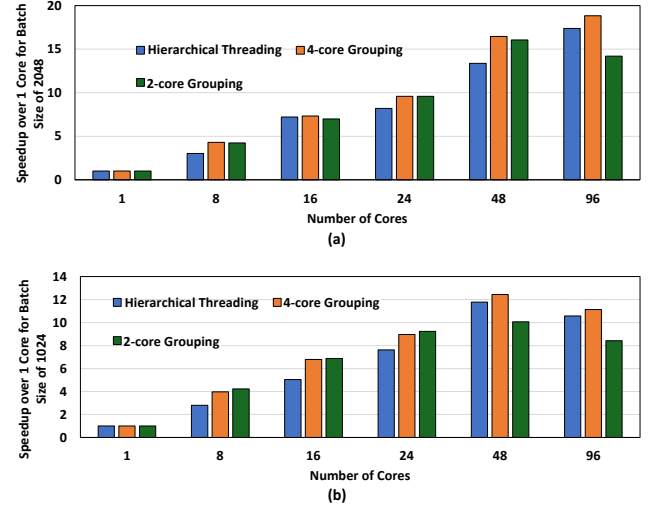


Figure 23. Speedup scaling comparison for a large model with 400 tables.

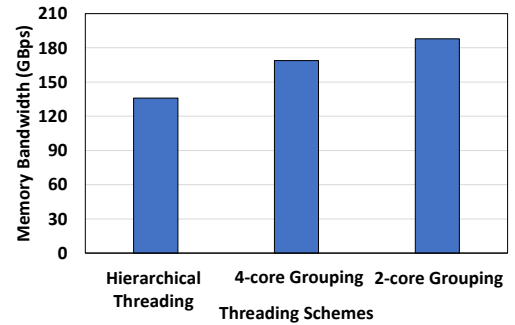


Figure 24. Memory bandwidth consumption of HT and Balance schedulers with 48 cores and 2048 batch size.

6 Discussion and Future Work

Applicability of Balance on non-AMD CPU-based SoCs:

We believe that Balance is applicable to CPU-based SoC architectures that do not group cores in CCDs. The idea of core grouping is independent of the CCD architecture and primarily depends on embedding table properties. The WMLP ordering metric depends on both table properties and L3 miss rates. The L3 miss rates can be estimated using either a logically shared or distributed L3 cache. The ordering may be different, depending on the L3 architecture, but the WMLP-based table ordering metric is still applicable. HWMLP versus LWMLP table partitioning is also independent of the CCD architecture because it maps all tables into 2 queues (as opposed to a number of queues matching the number of CCDs in HT). Work stealing is independent of the SoC architecture. Finally, the allocation of threads to cores in Balance is aware of the CCD core count and the isolation offered across L3 partitions. However, it can be modified to better suit a shared

L3 architecture. In one example, a Balance scheduler can virtually partition a shared L3 into fixed size clusters of N cores and schedule the tables to groups of consecutive, available cores to minimize inter-core communication latency and cross L3 NoC bandwidth.

Training: In DLRM training with GPUs, as shown in prior work [28, 39, 45, 51], communication overhead has been found to be the biggest performance bottleneck due to load imbalance. To resolve this, prior work has approximated the work per table with the pooling factor only. In contrast, as shown in Section 2.4, our TL heuristic more accurately captures the workload by accounting for off-chip memory accesses. Thus, for DLRM training, we expect our TL heuristics to balance table placement over a large pool of GPUs.

7 Related Works

Several prior works [9, 11, 12, 14, 16, 18, 20–27, 29, 40, 50] have studied and optimized DLRM inference, including DLRM inference on CPUs [11, 14, 21, 25, 40]. Jain et al. [21] proposed mapping each batch to a CPU core. However, the work per batch increases with larger models (containing more embedding tables and higher embedding dimensions) and data set sizes, thus making it difficult to meet real-time latency targets. Nair et al. [40] proposed parallelization of a batch using table threading (TT) and hierarchical threading (HT) and achieved significant performance improvements. However, as mentioned in Section 2.4, HT can suffer from scalability issues, which can be resolved using our proposed Balance scheduler.

Various prior works have developed accelerators [1, 9, 12, 18, 23, 24, 26, 27, 47, 49]. However, these proposals require custom silicon and demand considerable time and effort to bring to market, which makes it challenging to adopt them given the rapidly changing model parameters. Instead, Balance requires only software changes and runs on CPUs, thus making it much easier to deploy.

8 Conclusions

Optimizing DLRM inference for CPUs can provide significant value to data center efficiency as well as improve user experience. Therefore, understanding the inefficiencies in existing DLRM inference tasks is important. The heuristics and performance metrics developed in this work (TL heuristic and WMLP metric) have been successful in mitigating the inefficiencies. In addition, we demonstrate how load imbalance across threads and CCDs, suboptimal core allocation for embedding tables, and inefficient utilization of MLP resources can be addressed by our proposed mechanisms and techniques. We believe our analyses and proposed solutions can open up further opportunities not only in optimizing DLRM inference, but also workloads with similar irregular memory access patterns and MLP characteristics.

Acknowledgments

We extend our gratitude to the anonymous reviewers for their feedback and valuable insights, which have improved the paper. This research was partially funded by NSF grants {#1931531, #1955815, #1763681, #2116962, #2122155, #2028929, and #2211018}, and NSF Chameleon Cloud project CHI-231143. We extend special thanks to the members of AMD Research and Advanced Development and HPCL. AMD, the AMD Arrow logo, EPYC and combinations thereof are trademarks of Advanced Micro Devices, Inc. The OpenMP name and the OpenMP logo are registered trademarks of the OpenMP Architecture Review Board. PyTorch, the PyTorch logo, and any related marks are trademarks of The Linux Foundation. Ubuntu is a trademark of Canonical Ltd. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.

References

- [1] Muhammad Adnan, Yassaman Ebrahimzadeh Maboud, Divya Mahajan, and Prashant J Nair. 2024. Heterogeneous acceleration pipeline for recommendation system training. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1063–1079.
- [2] Amazon. 2024. Amazon Recommendation System. "<https://aws.amazon.com/personalize/>".
- [3] AMD. 2022. AMD Epyc 9654 CPU, code-named Genoa. "<https://www.amd.com/en/product/12191>".
- [4] Newsha Ardalani, Carole-Jean Wu, Zeliang Chen, Bhargav Bhushanam, and Adnan Aziz. 2022. Understanding scaling laws for recommendation models. *arXiv preprint arXiv:2208.08489* (2022).
- [5] Vivek M Bhasi, Aakash Sharma, Rishabh Jain, Jashwant Raj Gunasekaran, Ashutosh Pattnaik, Mahmut Taylan Kandemir, and Chita Das. 2024. Towards SLO-Compliant and Cost-Effective Serverless Computing on Emerging GPU Architectures. In *Proceedings of the 25th International Middleware Conference*. 211–224.
- [6] Sitian Chen, Haobin Tan, Amelie Chi Zhou, Yusen Li, and Pavan Balaji. 2024. UpDLRM: Accelerating Personalized Recommendation using Real-World PIM Architecture. *arXiv preprint arXiv:2406.13941* (2024).
- [7] Chips and Cheese. 2024. Ryzen-9950x. "<https://chipsandcheese.com/p/amds-ryzen-9950x-zen-5-on-desktop>".
- [8] Yujeong Choi, Jiin Kim, and Minsoo Rhu. 2024. ElasticRec: A Microservice-based Model Serving Architecture Enabling Elastic Resource Scaling for Recommendation Models. *arXiv preprint arXiv:2406.06955* (2024).
- [9] Amin Firoozshahian, Joel Coburn, Roman Levenstein, Rakesh Nattoji, Ashwin Kamath, Olivia Wu, Gurdeepak Grewal, Harish Aepala, Bhasker Jakka, Bob Dreyer, Adam Hutchin, Utku Diril, Krishnakumar Nair, Ehsan K. Aredestani, Martin Schatz, Yuchen Hao, Rakesh Komuravelli, Kunming Ho, Sameer Abu Asal, Joe Shajrawi, Kevin Quinn, Nagesh Sreedhara, Pankaj Kansal, Willie Wei, Dheepak Jayaraman, Linda Cheng, Pritam Chopda, Eric Wang, Ajay Bikumandla, Arun Karthik Sengottuvel, Krishna Thottampudi, Ashwin Narasimha, Brian Dodds, Cao Gao, Jiyuan Zhang, Mohammed Al-Sanabani, Ana Zehtabioskuie, Jordan Fix, Hangchen Yu, Richard Li, Kaustubh Gondkar, Jack Montgomery, Mike Tsai, Saritha Dwarakapuram, Sanjay Desai, Nili Avidan, Poorvaja Ramani, Karthik Narayanan, Ajit Mathews, Sethu Gopal, Maxim Naumov, Vijay Rao, Krishna Noru, Harikrishna Reddy, Prahlad Venkatapuram, and Alexis Bjorlin. 2023. MTIA: First Generation Silicon Targeting Meta's Recommendation Systems.

- In *Proceedings of the 50th Annual International Symposium on Computer Architecture* (Orlando, FL, USA) (ISCA '23). Association for Computing Machinery, New York, NY, USA, Article 80, 13 pages. doi:10.1145/3579371.3589348
- [10] Google. 2024. Google Q2 2024 Financial Report. "https://abc.xyz/assets/ae/e9/753110054014b6de4d620a2853f6/google-10-q-q2-2024.pdf".
- [11] Udit Gupta, Samuel Hsia, Vikram Saraph, Xiaodong Wang, Brandon Reagen, Gu-Yeon Wei, Hsien-Hsin S Lee, David Brooks, and Carole-Jean Wu. 2020. Deeprecsys: A system for optimizing end-to-end at-scale neural recommendation inference. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 982–995.
- [12] Udit Gupta, Samuel Hsia, Jeff Zhang, Mark Wilkening, Javin Pombra, Hsien-Hsin Sean Lee, Gu-Yeon Wei, Carole-Jean Wu, and David Brooks. 2021. RecPipe: Co-Designing Models and Hardware to Jointly Optimize Recommendation Quality and Performance. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (Virtual Event, Greece) (MICRO '21). Association for Computing Machinery, New York, NY, USA, 870–884. doi:10.1145/3466752.3480127
- [13] Udit Gupta, Carole-Jean Wu, Xiaodong Wang, Maxim Naumov, Brandon Reagen, David Brooks, Bradford Cottel, Kim Hazelwood, Mark Hempstead, Bill Jia, et al. 2020. The architectural implications of facebook's dnn-based personalized recommendation. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 488–501.
- [14] Udit Gupta, Carole-Jean Wu, Xiaodong Wang, Maxim Naumov, Brandon Reagen, David Brooks, Bradford Cottel, Kim Hazelwood, Mark Hempstead, Bill Jia, Hsien-Hsin S. Lee, Andrey Malevich, Dheevatsa Mudigere, Mikhail Smelyanskiy, Liang Xiong, and Xuan Zhang. 2020. The Architectural Implications of Facebook's DNN-Based Personalized Recommendation. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 488–501. doi:10.1109/HPCA47549.2020.00047
- [15] GVR. 2024. Recommendation Engine Market Size. "https://www.grandviewresearch.com/industry-analysis/recommendation-engine-market-report".
- [16] Samuel Hsia, Udit Gupta, Mark Wilkening, Carole-Jean Wu, Gu-Yeon Wei, and David Brooks. 2020. Cross-stack workload characterization of deep recommendation systems. In *2020 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 157–168.
- [17] Hulu. 2024. Hulu Recommendation System. https://help.hulu.com/article/hulu-personalized-recommendations#:~:text=While%20you're%20looking%20for,get%20to%20know%20you%20better.
- [18] Ranggi Hwang, Taehun Kim, Youngeun Kwon, and Minsoo Rhu. 2020. Centaur: A chiplet-based, hybrid sparse-dense accelerator for personalized recommendations. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 968–981.
- [19] Intel. 2024. Intel Extension For Pytorch. "https://github.com/intel/intel-extension-for-pytorch".
- [20] Rishabh Jain, Vivek M Bhasi, Adwait Jog, Anand Sivasubramaniam, Mahmut T Kandemir, and Chita R Das. 2024. Pushing the Performance Envelope of DNN-based Recommendation Systems Inference on GPUs. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1217–1232.
- [21] Rishabh Jain, Scott Cheng, Vishwas Kalagi, Vrushabh Sanghavi, Samvit Kaul, Meena Arunachalam, Kiwan Maeng, Adwait Jog, Anand Sivasubramaniam, Mahmut Taylan Kandemir, and Chita R. Das. 2023. Optimizing CPU Performance for Recommendation Systems At-Scale. In *Proceedings of the 50th Annual International Symposium on Computer Architecture* (Orlando, FL, USA) (ISCA '23). Association for Computing Machinery, New York, NY, USA, Article 77, 15 pages. doi:10.1145/3579371.3589112
- [22] Gopi Krishna Jha, Anthony Thomas, Nilesh Jain, Sameh Gobriel, Tajana Rosing, and Ravi Iyer. 2024. Mem-Rec: Memory Efficient Recommendation System using Alternative Representation. In *Asian Conference on Machine Learning*. PMLR, 518–533.
- [23] Hongju Kal, Seokmin Lee, Gun Ko, and Won Woo Ro. 2021. SPACE: Locality-Aware Processing in Heterogeneous Memory for Personalized Recommendations. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 679–691. doi:10.1109/ISCA52012.2021.00059
- [24] Liu Ke, Udit Gupta, Benjamin Youngjae Cho, David Brooks, Vikas Chandra, Utku Diril, Amin Firoozshahian, Kim Hazelwood, Bill Jia, Hsien-Hsin S. Lee, Meng Li, Bert Maher, Dheevatsa Mudigere, Maxim Naumov, Martin Schatz, Mikhail Smelyanskiy, Xiaodong Wang, Brandon Reagen, Carole-Jean Wu, Mark Hempstead, and Xuan Zhang. 2020. RecNMP: Accelerating Personalized Recommendation with near-Memory Processing. In *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture* (Virtual Event) (ISCA '20). IEEE Press, 790–803. doi:10.1109/ISCA45697.2020.00070
- [25] L. Ke, U. Gupta, M. Hempstead, C. Wu, H. S. Lee, and X. Zhang. 2022. Hercules: Heterogeneity-Aware Inference Serving for At-Scale Personalized Recommendation. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 141–154. doi:10.1109/HPCA53966.2022.00019
- [26] Liu Ke, Xuan Zhang, Jinin So, Jong-Geon Lee, Shin-Haeng Kang, Sukhan Lee, Songyi Han, YeonGon Cho, Jin Hyun Kim, Yongsuk Kwon, KyungSoo Kim, Jin Jung, Ilkwon Yun, Sung Joo Park, Hyunsun Park, Joonho Song, Jeonghyeon Cho, Kyomin Sohn, Nam Sung Kim, and Hsien-Hsin S. Lee. 2022. Near-Memory Processing in Action: Accelerating Personalized Recommendation With AxDIMM. *IEEE Micro* 42, 1 (2022), 116–127. doi:10.1109/MM.2021.3097700
- [27] Youngeun Kwon, Yunjae Lee, and Minsoo Rhu. 2019. Tensordimm: A practical near-memory processing architecture for embeddings and tensor operations in deep learning. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 740–753.
- [28] Fan Lai, Wei Zhang, Rui Liu, William Tsai, Xiaohan Wei, Yuxi Hu, Sabin Devkota, Jianyu Huang, Jongsoo Park, Xing Liu, Zeliang Chen, Ellie Wen, Paul Rivera, Jie You, Chun cheng Jason Chen, and Mosharaf Chowdhury. 2023. {AdaEmbed}: Adaptive Embedding for {Large-Scale} Recommendation Models. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, Boston, MA, 817–831. https://www.usenix.org/conference/osdi23/presentation/lai
- [29] Yejin Lee, Seong Hoon Seo, Hyunji Choi, Hyoung Uk Sul, Soosung Kim, Jae W. Lee, and Tae Jun Ham. 2021. MERCI: Efficient Embedding Reduction on Commodity Hardware via Sub-Query Memoization. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Virtual, USA) (ASPLOS '21). Association for Computing Machinery, New York, NY, USA, 302–313. doi:10.1145/3445814.3446717
- [30] Kiran Kumar Matam, Hani Ramezani, Fan Wang, Zeliang Chen, Yue Dong, Maomao Ding, Zhiwei Zhao, Zhengyu Zhang, Ellie Wen, and Assaf Eisenman. 2024. {QuickUpdate}: a {Real-Time} Personalization System for {Large-Scale} Recommendation Models. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 731–744.
- [31] Meta. 2023. Embedding lookup Production dataset. "https://github.com/facebookresearch/dlrm_datasets".
- [32] Meta. 2024. DLRM using PyTorch. "https://github.com/facebookresearch/dlrm".
- [33] Meta. 2024. Facebook Recommendation System. "https://ai.meta.com/blog/ai-unconnected-content-recommendations-facebook-instagram/".
- [34] Meta. 2024. Instagram Recommendation System. "https://engineering.fb.com/2023/08/09/ml-applications/scaling-instagram-explore-recommendations-system/".
- [35] Meta. 2024. Meta Q1 2024 Financial Report. "https://s21.q4cdn.com/399680738/files/doc_financials/2024/q1/Earnings-

- Presentation-Q1-2024.pdf".
- [36] Meta. 2024. Multi-threading in PyTorch. "https://pytorch.org/docs/stable/notes/cpu_threading_torchscript_inference.html".
 - [37] Meta. 2024. PyTorch DLRM. "<https://github.com/facebookresearch/dlrm/>".
 - [38] Meta. 2024. PyTorch Embedding Bag Operator. "<https://github.com/pytorch/pytorch/blob/main/aten/src/ATen/native/EmbeddingBag.cpp>".
 - [39] Dheevatsa Mudigere, Yuchen Hao, Jianyu Huang, Zhihao Jia, Andrew Tulloch, Srinivas Sridharan, Xing Liu, Mustafa Ozdal, Jade Nie, Jongsoo Park, Liang Luo, Jie (Amy) Yang, Leon Gao, Dmytro Ivchenko, Aarti Basant, Yuxi Hu, Jiyan Yang, Ehsan K. Ardestani, Xiaodong Wang, Rakesh Komuravelli, Ching-Hsiang Chu, Serhat Yilmaz, Huayu Li, Jiyuan Qian, Zhuobo Feng, Yinbin Ma, Junjie Yang, Ellie Wen, Hong Li, Lin Yang, Chonglin Sun, Whitney Zhao, Dimitry Melts, Krishna Dhulipala, KR Kishore, Tyler Graf, Assaf Eisenman, Kiran Kumar Matam, Adi Gangidi, Guoqiang Jerry Chen, Manoj Krishnan, Avinash Nayak, Krishnakumar Nair, Bharath Muthiah, Mahmoud khorashadi, Pallab Bhattacharya, Petr Lapukhov, Maxim Naumov, Ajit Mathews, Lin Qiao, Mikhail Smelyanskiy, Bill Jia, and Vijay Rao. 2022. Software-hardware co-design for fast and scalable training of deep learning recommendation models. In *Proceedings of the 49th Annual International Symposium on Computer Architecture* (New York, New York) (ISCA '22). Association for Computing Machinery, New York, NY, USA, 993–1011. doi:10.1145/3470496.3533727
 - [40] Krishnakumar Nair, Avinash-Chandra Pandey, Siddappa Karabannavar, Meena Arunachalam, John Kalamatianos, Varun Agrawal, Saurabh Gupta, Ashish Sirasao, Elliott Delaye, Steve Reinhardt, et al. 2024. Parallelization Strategies for DLRM Embedding Bag Operator on AMD CPUs. *IEEE Micro* (2024).
 - [41] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G. Azzolini, Dmytro Dzhulgakov, Andrey Malleevich, Ilia Cherniavskii, Yinghai Lu, Raghuraman Krishnamoorthi, Ansha Yu, Volodymyr Kondratenko, Stephanie Pereira, Xianjie Chen, Wenlin Chen, Vijay Rao, Bill Jia, Liang Xiong, and Misha Smelyanskiy. 2019. Deep Learning Recommendation Model for Personalization and Recommendation Systems. arXiv:1906.00091 [cs.IR]
 - [42] Netflix. 2024. Netflix Recommendation System. "<https://research.netflix.com/research-area/recommendations>".
 - [43] Taehyung Park, Seungjin Yang, Jongmin Seok, Hyuk-Jae Lee, Juhyun Kim, and Chae Eun Rhee. 2024. Accelerating Large-Scale DLRM Inference through Dynamic Hot Data Rearrangement. In *2024 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 1–5.
 - [44] SemiAnalysis. 2024. The Memory Wall. "<https://www.semianalysis.com/p/the-memory-wall>".
 - [45] Geet Sethi, Bilge Acun, Niket Agarwal, Christos Kozyrakis, Caroline Trippel, and Carole-Jean Wu. 2022. RecShard: Statistical Feature-Based Memory Optimization for Industry-Scale Neural Recommendation. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) (ASPLOS '22). Association for Computing Machinery, New York, NY, USA, 344–358. doi:10.1145/3503222.3507777
 - [46] Yian Su, Mike Rainey, Nick Wanninger, Nadharm Dhantravan, Jasper Liang, Umut A Acar, Peter Dinda, and Simone Campanoni. 2024. Compiling Loop-Based Nested Parallelism for Irregular Workloads. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 232–250.
 - [47] Xuan Sun, Hu Wan, Qiao Li, Chia-Lin Yang, Tei-Wei Kuo, and Chun Jason Xue. 2022. Rm-ssd: In-storage computing for large-scale recommendation inference. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 1056–1070.
 - [48] TikTok. 2024. TikTok Recommendation System. "<https://www.tiktok.com/transparency/en-us/recommendation-system/>".
 - [49] Mark Wilkening, Udit Gupta, Samuel Hsia, Caroline Trippel, Carole-Jean Wu, David Brooks, and Gu-Yeon Wei. 2021. RecSSD: Near Data Processing for Solid State Drive Based Recommendation Inference. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Virtual, USA) (ASPLOS '21). Association for Computing Machinery, New York, NY, USA, 717–729. doi:10.1145/3445814.3446763
 - [50] Haojie Ye, Sanketh Vedula, Yuhang Chen, Yichen Yang, Alex Bronstein, Ronald Dreslinski, Trevor Mudge, and Nishil Talati. 2023. GRACE: A Scalable Graph-Based Approach to Accelerating Recommendation Model Inference. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (Vancouver, BC, Canada) (ASPLOS 2023). Association for Computing Machinery, New York, NY, USA, 282–301. doi:10.1145/3582016.3582029
 - [51] Daochen Zha, Louis Feng, Qiaoyu Tan, Zirui Liu, Kwei-Herng Lai, Bhargav Bhushanam, Yuandong Tian, Arun Kejariwal, and Xia Hu. 2022. Dreamshard: Generalizable embedding table placement for recommender systems. *Advances in Neural Information Processing Systems* 35 (2022), 15190–15203.
 - [52] Qin Zhao, Ioana Cutcutache, and Weng-Fai Wong. 2010. Pipa: Pipelined profiling and analysis on multicore systems. *ACM Transactions on Architecture and Code Optimization (TACO)* 7, 3 (2010), 1–29.
 - [53] Mai Zheng, Vignesh T Ravi, Wenjing Ma, Feng Qin, and Gagan Agrawal. 2012. Gmprof: A low-overhead, fine-grained profiling approach for gpu programs. In *2012 19th International Conference on High Performance Computing*. IEEE, 1–10.
 - [54] Yuhong Zhong, Daniel S Berger, Carl Waldspurger, Ishwar Agarwal, Rajat Agarwal, Frank Hady, Karthik Kumar, Mark D Hill, Mosharaf Chowdhury, and Asaf Cidon. 2024. Managing Memory Tiers with CXL in Virtualized Environments. In *Symposium on Operating Systems Design and Implementation*.